

# **Competitive Programming**

**Section 7: Foundation mixed contest**

Competitive Programming Course

2026-07-04

# Table of contents

|    |                                                 |    |
|----|-------------------------------------------------|----|
| 1  | Why a mixed contest now                         | 3  |
| 2  | Contest triage                                  | 4  |
| 3  | Problem A signal: frequencies plus sorting      | 5  |
| 4  | Problem B signal: range updates                 | 6  |
| 5  | Problem C signal: nonnegative contiguous streak | 7  |
| 6  | Technique selection diagram                     | 8  |
| 7  | Reading tie-breaking rules                      | 9  |
| 8  | Upsolving workflow                              | 10 |
| 9  | Common mixed-contest mistakes                   | 11 |
| 10 | Before you submit                               | 12 |
| 11 | Exercises                                       | 13 |

# 1 Why a mixed contest now

Sections 1 through 6 introduced the foundation tools:

- constraints and I/O;
- simulation and brute force;
- counting and frequency maps;
- prefix sums and difference arrays;
- sorting;
- two pointers and sliding windows.

Real contests do not label problems with the technique. A mixed section trains the missing skill: deciding which tool the statement is asking for.

This section should be attempted like a short virtual contest. Read all three problems first, estimate the intended complexity, solve the easiest one, and then upsolve carefully.

## 2 Contest triage

When you open a set, do a quick pass:

```
for each problem:
    read statement and constraints
    identify input size and value range
    guess one or two possible tools
    mark confidence: high / medium / low
```

Do not spend 25 minutes on the first problem just because it is first. A contest is a scheduling problem too.

For foundation problems, the technique clues are often visible:

| Statement shape                     | Likely tool                              |
|-------------------------------------|------------------------------------------|
| many equal values or distinct types | counting/frequencies                     |
| many range updates                  | difference array                         |
| contiguous segment under a limit    | sliding window if values are nonnegative |
| closest or ordered relation         | sorting                                  |
| small explicit search space         | brute force                              |

### 3 Problem A signal: frequencies plus sorting

Inventory cleanup asks:

delete items so that at most  $k$  distinct colors remain.

The values themselves do not matter except for equality. That points to a frequency table.

Once you know each color's frequency, the question becomes:

which color groups should be deleted entirely?

To reduce the number of distinct colors with minimum deletions, delete the smallest groups first. That points to sorting the frequencies.

Technique stack:

```
count colors -> sort frequencies -> remove smallest groups
```

## 4 Problem B signal: range updates

Peak load says each job contributes to every integer time in  $[l, r]$ .

The direct simulation is:

```
for every job:
    for every time in its interval:
        add load
```

That can be too slow. The phrase “add to every position in a range” should trigger the difference-array pattern:

```
diff[l] += w
diff[r + 1] -= w
```

Then a single scan reconstructs the load and finds the earliest maximum.

## 5 Problem C signal: nonnegative contiguous streak

Best study streak asks for a contiguous segment with sum at most  $k$ , and the values are nonnegative.

That is the sliding-window condition from Section 6. The added contest detail is tie-breaking:

among maximum-length streaks, choose the earliest start.

So the answer update must compare:

```
larger length first  
then smaller start
```

## 6 Technique selection diagram

```
Does order matter?  
|  
+-- no, only equality -> frequency table  
|  
+-- yes, contiguous range?  
    |  
    +-- many fixed range updates -> difference array  
    |  
    +-- nonnegative sum limit -> sliding window  
    |  
    +-- arbitrary signs -> prefix sums / maps
```

This diagram is not universal. It is a contest-time checklist for the tools covered so far.

## 7 Reading tie-breaking rules

Tie-breaking rules are part of the problem, not output decoration.

Examples:

- minimum deletions;
- earliest time of maximum load;
- earliest start among longest streaks.

For each answer variable, ask:

```
What is the primary objective?  
What is the secondary objective?  
What should happen if everything ties?
```

In Problem B, the primary objective is maximum load. The secondary objective is earliest time. That means update only on strictly larger load during a left-to-right scan; the first occurrence remains stored.

In Problem C, update on a longer streak, and for equal length update only if the start is earlier.

## 8 Upsolving workflow

After attempting the section:

1. Reproduce the failure locally.
2. Identify whether the issue is technique choice, implementation, or edge case.
3. Write or inspect a slow oracle for small cases.
4. Add one targeted test that would have caught the bug.
5. Only then read the full editorial solution.

The local random generators in this course intentionally use slow oracles for small cases. This is how many of the hand-written sample mistakes were caught while building the course.

## 9 Common mixed-contest mistakes

- Recognizing a familiar tool but ignoring a new tie-breaker.
- Using a sliding window when negative values are allowed.
- Sorting values when only frequencies matter.
- Applying range updates directly even though  $n + m$  is large.
- Solving the first problem slowly while easier points are available later.
- Trusting hand-written samples without an oracle.

## 10 Before you submit

For each problem, write one sentence:

```
The controlling constraint is ...  
The intended complexity is ...  
The key invariant is ...  
The edge case I tested is ...
```

If you cannot fill these in, the solution is not ready.

# 11 Exercises

Attempt these as a timed set:

1. `a_inventory_cleanup`: counting plus sorting.
2. `b_peak_load`: difference array plus earliest maximum.
3. `c_best_study_streak`: sliding window plus tie-breaking.

Run:

```
python3 sections/07_foundation_mixed_contest/check.py
```